KQL 101: An introduction to Kusto Query Languge

https://www.youtube.com/@TenMinuteKQL

Table Name	Description	Example
🔐 AuthenticationEvents	Logins to devices (successes and failures).	When someone tries to log in to their email account.
📧 Email	Emails sent and received by employees.	When an employee sends or receives a work email.
👥 Employees	Info about the company's employees.	Names, job titles, and contact details of employees.
📄 FileCreationEvents	Files created on employee devices.	When someone saves a new document on their computer.
🌐 InboundNetworkEvents	Internet activity coming into the company network.	When someone visits the company website.
🚀 OutboundNetworkEvents	Internet activity going out from the company network.	When someone visits an external website like google.com.
🌍 PassiveDns (External)	Which IP addresses match up with which websites.	Figuring out that "google.com" matches with IP "8.8.8.8".
💻 ProcessEvents	Programs that start running on employee devices.	When an employee opens a program like Microsoft Word.
⚠️ SecurityAlerts	Security warnings from devices or email systems.	When antivirus software detects a virus on a computer.

Table very useful to be translate to other sequel language

In cybersecurity, every log tells a story. Failed logins, unusual network activity, and hidden threats are all buried in massive datasets. Kusto Query Language (KQL) is a powerful tool that helps you efficiently search, filter, and analyze logs to uncover security incidents before they escalate.

In this module, you'll learn how to use KQL to query logs, identify patterns, and think like an investigator to stay ahead of cyber threats. Whether you're a security analyst, threat hunter, or just getting started, KQL 101 will equip you with the skills to turn raw data into actionable intelligence.

What You'll Learn
Understand the data contained in tables and how to navigate columns and rows in KQL.
Learn how to filter and refine data using commands like where, contains, and has.
Use logical operators like and, or, and not to build precise queries.
Write efficient KQL queries to search, sort, and extract relevant security data.

KQL is a powerful, easy-to-learn query language designed for fast, flexible data exploration in Azure Data Explorer (ADX). Security analysts, incident responders, threat intelligence analysts, and threat hunters use it every day to:

Search logs in seconds—because speed matters when investigating threats
Filter and analyze events—cut through the noise and focus on what's important

Identify anomalies and patterns—spot unusual activity before it becomes a bigger problem

- □ Navigate and explore datasets
- □ Filter and refine security logs
- □ Extract meaningful insights from massive amounts of data
- □ Apply KQL in real-world cybersecurity scenarios

```
Employees
| count
```

`Distinct`

```
1. Identify unique IP addresses that have accessed a system
2. List all different usernames that have attempted to log in
3. Find the variety of processes running on a machine
Employees
| distinct role
```

```
Email
| where recipient == "noemi_tep@jojoshospital.org"
| where sender !contains "jojoshospital"
```

```
AuthenticationEvents
| where src_ip == "10.10.0.144" and result == "Failed Login"
```

How many results do we get in AuthenticationEvents for these two IP addresses?

```
AuthenticationEvents
| where src_ip == "10.10.0.144" or src_ip == "10.10.0.86"
```

How many `failed` login attempts were from these IP addresses?

```
AuthenticationEvents
| where src_ip == "10.10.0.144"
or src_ip == "10.10.0.86"
or src_ip == "10.10.0.86"
or src_ip == "10.10.0.20"
or src_ip == "10.10.0.109"

AuthenticationEvents
| where src_ip in ("10.10.0.144", "10.10.0.86", "10.10.0.86", "10.10.0.20",
"10.10.0.109")

AuthenticationEvents
| where src_ip in ("10.10.0.144", "10.10.0.86", "10.10.0.86", "10.10.0.20",
"10.10.0.109") and result == "Failed Login"
```

How many distinct websites(urls) did Nancy Roberts visit?

```
Employees
| where name == "Nancy Roberts"

OutboundNetworkEvents
| where src_ip == "10.10.0.30"
| distinct url
```

n KQL, you can add comments using `//`. Anything written after `//` is ignored when the query runs, so you can use it to document findings or remind yourself what a query does.

```
// Finding Nancy Roberts' IP address to check web browsing
Employees
| where name == "Nancy Roberts"
//Nancy's ip address: 10.10.0.30

// Find all IT support employees
Employees
| where role contains "IT support"

// Look for logins from IPs used by IT support
AuthenticationEvents
| where src_ip in ("10.10.0.75", "10.10.0.42", "10.10.0.34", "10.10.0.10", "10.10.0.2")


// Count all failed logins
AuthenticationEvents
| where result == "Failed Login"
| count
```

How many websites did employees with the first name `Mary` visit?

```
Methode 1

// search employe with first name mary
Employees
| where name contains "Mary"

OutboundNetworkEvents
| where src_ip in ("10.10.0.84", "10.10.0.169", "10.10.0.161", "10.10.0.24",
"10.10.0.125", "10.10.0.113", "10.10.0.103")
| count

Methode 2

let mary_ips = Employees
| where name has "Employee Name"
| distinct ip_addr;
OutboundNetworkEvents
| where src_ip in (mary_ips)

1. We use a `let` statement to create a temporary variable called `mary_ips`.
2. This variable stores the `IP addresses` of employees whose name contains "Mary"
(found in the `Employees` table).
3. We then use `mary_ips` to filter the `OutboundNetworkEvents` table, checking how
many websites those employees visited.
```

```
let mary_ips = Employees
| where name has "Mary"
| distinct ip_addr;
OutboundNetworkEvents
| where src_ip in (mary_ips)
```

How many authentication attempts did we see to the accounts of employees with the first name Mary?

```
let Mary_name = Employees
| where name has "Mary"
| distinct username;
AuthenticationEvents
| where username in (Mary_name)
| count
```